

Abstract

This report reproduces and extends the central claim of Leviathan, Kalman, and Matias (arXiv:2512.14982): repeating a user’s input prompt verbatim improves non-reasoning LLM accuracy without increasing output length or latency. Testing was restricted to ARC-Challenge and NameIndex on current Gemini-family models, as the paper’s primary Gemini models were unavailable at the time of testing.

The core claim holds true: prompt repetition produces strongly significant accuracy gains on NameIndex using the gemini-3.6-flash model as the primary model with repetition ceilings near 97-98% and no measurable increase in latency. The effect generalizes under Mandarin task instructions, multiple repetition counts, and four of five additional current Gemini models. ARC results are directionally consistent but do not reach significance at n=100.

1 What The Paper Claims

Leviathan, Kalman, and Matias (arXiv:2512.14982, December 2025) proposed that repeating a user's input prompt verbatim (transforming <QUERY> into <QUERY><QUERY>) improves the accuracy of non-reasoning large language models across a range of benchmark tasks, without increasing the number of generated output tokens or measurable inference latency. The main theoretical reason is that causal language models cannot have earlier tokens attend to later tokens, and so the ordering of context within a query can affect prediction quality. Prompt repetition addresses this constraint by allowing tokens in the first instance of the prompt to attend to all tokens in the second prompt, and vice versa, which gives every token full bidirectional context without requiring a change to the model architecture or decoding procedure. The authors frame this as a “parallelizable prefill-stage” operation and distinguish it from reason-inducing techniques such as chain-of-thought (CoT), which increases both output length and latency.

The paper tested this claim across seven models in February and March of 2025, including Gemini 2.0 Flash, Gemini 2.0 Flash Lite, GPT-4o, GPT-4o-mini, Claude 3 Haiku, Claude 3.7 Sonnet, and Deepseek V3, all of which were queried via official API providers. Benchmarks used include the ARC-Challenge, OpenBookQA, GSM8K, MMLU-Pro, MATH, and two custom tasks: NameIndex (retrieve the 25th name from a list of 50) and MiddleMatch (identify the name

between two anchors in a 40-element list with repeated elements). For benchmarks involving multiple-choice questions, results are reported in two orders: question-first (question precedes answer choices) and options-first (answer choices precede the question). The authors expected options-first to produce lower baseline accuracy, since the model processes answer choices before seeing the question. Statistical significance was evaluated using McNemar’s test with a significance level of 0.1, but the paper does not apply multiple comparison correction across the 70 benchmark-model combinations tested (see Section 5.9).

The headline result is that prompt repetition wins 47 of 70 benchmark-model combinations with zero losses under the $p < 0.1$ threshold. Gains are smaller for question-first multiple-choice tasks, larger for options-first, and largest for the two custom tasks. The authors' main illustrative example is Gemini 2.0 Flash Lite on NameIndex, which improves from a baseline 21.33% accuracy to a 97.33% accuracy (a result this reproduction uses as its direct comparison benchmark). The paper also tests three variations beyond vanilla repetition: Prompt Repetition (Verbose), which inserts “Let me repeat that:” between the two queries; Prompt Repetition $\times 3$, which adds a third copy with a similar connector; and a padding control, which replaces the second copy with period characters of equivalent length. The $\times 3$ variant is reported to often substantially outperform vanilla repetition on NameIndex and MiddleMatch. The padding control does not improve performance, which the authors take as evidence that the gains come from the semantic content of the repeated prompt, not from increased input length alone. Finally, when models are instructed to reason step-by-step, prompt repetition is neutral to slightly positive (5 wins, 1 loss, 22 ties), which the authors attribute to reasoning models already replicating part of the repetition effect internally through their CoT output.

2 What I Reproduced And What I Didn’t

This reproduction targets two of the paper’s seven benchmarks: ARC-Challenge and NameIndex, selected as representative cases of multiple-choice and positional retrieval tasks respectively. The remaining benchmarks (OpenBookQA, GSM8K, MMLU-Pro, MATH) and MiddleMatch task were not implemented, as this reproduction focuses narrowly on the two conditions most directly comparable to the paper’s primary result. Testing was restricted to Gemini-family models, as the

paper’s headline example, and the only result with a published exact accuracy figure, was on Gemini 2.0 Flash Lite. Both accuracy and latency were logged per question.

The ARC-Challenge was loaded from the AllenAI HuggingFace repository, the standard distribution of the dataset cited in the paper, and tested in both types of orders, and at $n=100$ as the paper did not specify a sample size. The NameIndex task structure was reproduced exactly. Each question presents a unique list of 50 randomly generated names, fixed across runs via a constant seed.

The prompt formats were matched to the templates specified in the paper’s Appendix A.4: vanilla repetition repeats <QUERY> with a newline separator. The x3 variant follows the paper’s exact verbose template. The paper states that it asked models “not to reason” but does not publish the exact instructions used. The reproduction implements that condition with the system prompt “Output only the name. No explanation, no punctuation, no list number, no position, just the name.” for NameIndex and “Do not explain your reasoning. Answer directly and concisely.” for the ARC-Challenge. Both prompts were used identically in both the baseline and repetition conditions. This may not replicate the paper’s condition fully, since the paper does not publish its own instruction. Temperature is set to 0 for reproducibility, although the value wasn’t specified in the paper. McNemar’s test is applied two-sided with $p < 0.1$, matching the paper’s criteria. The paper does not state whether it used a one-sided or two-sided test; two-sided was chosen as the more conservative interpretation.

Under a failure taxonomy (See Appendix A), responses were classified as ERROR (API failure, 0.0s latency, no model response), BLOCKED (model generated output but returned no parseable answer), EMPTY (extractor returned no name), LEAKED (raw output continued past the answer into subsequent list items), or a parsed answer. BLOCKED, EMPTY, and LEAKED were scored as incorrect, as in each case the model received the prompt and generated tokens. ERROR responses never reached the model and were excluded from both conditions, so baseline and repetition are always scored over an identical question set. In the runs reported, ERRORS occurred only in two runs of the repetition-count experiment, where a single ERROR in each reduces the denominator to 74; all other denominators are 75. A full definition of each category is given in Appendix A. Run selection followed a single pre-specified rule: a run was excluded if

either condition produced five or more combined ERROR and BLOCKED responses, and the first three runs passing that threshold were used as the reported result. This rule excluded gemini-3.7-flash runs 1-3 and no others; every other model passed on its first three runs.

Gemini 2.0 Flash Lite, the paper’s primary NameIndex model, was unavailable via the APIs as of August 2026. This reproduction uses gemini-3.6-flash as the primary model. Additional runs were conducted on gemini-3.1-flash-lite, gemini-3.5-flash-lite, gemini-3.5-flash, gemini-3.7-flash, and gemini-3.8-flash to examine how the effect varies across model generations.

3 My Results vs. The Paper’s Results

3.1 Accuracy

<u>Condition</u>	<u>Paper Baseline</u>	<u>My Baseline</u>	<u>Paper Repetition</u>	<u>My Repetition</u>	<u>Δ</u>	<u>p-value</u>	<u>Reproduced?</u>
ARC question-first	Not reported	87.00% (87/100)	Not reported	93.00% (93/100)	+6.00%	0.1094	Directionally, not significant
ARC options-first	Not reported	84.00% (84/100)	Not reported	90.00% (90/100)	+6.00%	0.2632	Directionally, not significant
NameIndex	21.33%	Mean: 68.89% Range: 66.67% - 72.00%	97.33%	Mean: 97.78% Range: 97.33% - 98.67%	+28.89%	Run 1: 2.10×10^{-5} Run 2: 8.05×10^{-7} Run 3: 2.98×10^{-6}	Partially, ceiling matches but baseline does not

The ARC results are directionally consistent with the paper. Both orderings show a +6% improvement with repetition, but neither reaches statistical significance with $p < 0.1$ at $n=100$, likely due to ceiling effects of a stronger model. The paper does not report exact ARC accuracy figures for Gemini 2.0 Flash Lite specifically, so a direct numerical comparison is not possible (see Section 5.1). The most likely explanation for the lack of significance is that the baseline accuracy is already high (87% question-first, 84% options-first), leaving limited room for improvement in repetition accuracy. The McNemar test for question-first yields $p=0.1094$ with $b=2$, $c=8$; options-first yields $p=0.2632$ with $b=7$, $c=13$. In both cases the discordant pair counts are too small to reach significance, though the direction favors repetition in both.

Unlike ARC, NameIndex allows a direct numerical comparison to the paper's reported figures. The repetition ceiling accuracy of 97.78% matches closely to the paper's reported figure for Gemini 2.0 Flash Lite. However, the baseline accuracy does not: the paper reports 21.33%, while this reproduction observes a 68.89% mean on gemini-3.6-flash. This result is expected, since gemini-3.6-flash is a substantially stronger model than gemini-2.0-flash-lite, and the stronger model has better positional retrieval capabilities, which retrieves the correct name more often without prompt manipulation. The smaller absolute improvement in accuracy is a direct consequence of the higher baseline the newer model provides, and this finding is consistent with the paper's observation that stronger models show smaller improvements from repetition. The McNemar test for NameIndex yields three different b/c values: $b=1$ $c=20$ for run 1, $b=1$ $c=25$ for run 2, and $b=1$, $c=23$ for run 3. The three NameIndex runs were all strongly significant.

3.2 Latency

The paper claims prompt repetition improves accuracy without increasing inference latency. This claim is tested here directly, since per-question wall-clock latency was logged for every API call. Combining NameIndex and ARC-Challenge on gemini-3.6-flash gives 424 baseline and 425 repetition calls, one baseline call that recorded a latency of 0.0 seconds was excluded. The median latency was 2.60 seconds for baseline and 2.54 seconds for repetition; the means were 5.99 and 5.44 seconds. Doubling the prompt therefore produced no measurable latency penalty, consistent with the paper's claim that repetition is a prefill-stage operation that does not extend generation.

Latency measured this way includes network round-trip time and provider-side queuing, so it is a noisier measure than the token counts the paper's claim ultimately concerns. Output token counts were not logged in this reproduction, so the “without increasing output length” half of the claim remains untested here. The NameIndex task expects a single name as an output, which makes a large divergence in output length unlikely, but that is only an assumption.

4 Additional Experiments

4.1 Mandarin Instructioning

<u>Run</u>	<u>Baseline</u>	<u>Repetition</u>	<u>b</u>	<u>c</u>	<u>p-value</u>	<u>Leaked</u> (Baseline → Repetition)
1	66.67% (50/75)	96.00% (72/75)	3	25	2.75×10^{-5}	17 → 2
2	73.33% (55/75)	93.33% (70/75)	4	19	0.0026	10 → 3
3	72.00% (54/75)	92.00% (69/75)	5	20	0.0041	9 → 3
Mean	70.67%	93.78%				
Range	66.67% - 73.33%	92.00% - 96.00%				

This experiment tests whether the repetition effect is language-agnostic or specific to English token processing. The NameIndex prompt was translated to Mandarin (“以下是一个名字列表...第25个名字是什么？”) while keeping the names in English, and run across the same 75 questions and experimental setup as the primary experiment.

The experiment was repeated three times. Across these runs, the Mandarin instruction condition produced a mean baseline accuracy of 70.67% (range: 66.67% - 73.33%) and a mean repetition accuracy of 93.78% (range: 92.00% - 96.00%). Compared to the English primary experiment

(baseline: 68.89%, repetition: 97.78%), the Mandarin instruction baselines had slight deviations but were nearly identical, and the Mandarin instruction repetitions were slightly lower. The effect was strongly significant across all three trials.

The failure taxonomy provides a suggestion why the Mandarin repetition ceiling sits lower. Under English instructions, gemini-3.6-flash produced 44 leaked responses in the baseline condition and 3 under repetition across three runs. Under Mandarin instructions the same model produced 36 and 8. Repetition suppresses list-continuation behaviour less effectively when the instruction is in Mandarin, and the residual leakage induces a lower ceiling (93.78% versus 97.78%). The instruction language therefore appears to affect how reliably the model switches out of list-continuation mode, rather than affecting positional retrieval itself.

It is important to recognize that even though the instruction language differed, the names remain in English in both conditions, meaning this tests instruction language only, not full cross-lingual transfer. Whether the effect holds when both the instruction and the names are in a non-Latin script remains an open question.

4.2 Repetition Count Scaling

The paper tests 2× and 3× repetition and reports that 3× “often substantially outperforms” 2× on NameIndex, and lists further exploration of repetition count as a future direction. This experiment extends that analysis by testing all of the following four counts on gemini-3.6-flash: 1x, 2× verbose, 3× verbose, and 4× verbose (custom), using the paper’s exact verbose templates if applicable. The 1× and 2× non-verbose conditions were retained as a point of reference. All verbose conditions were run three times.

<u>Repetitions</u>	<u>Template</u>	<u>Mean Accuracy</u>	<u>Range</u>
1× (baseline)	None	68.89%	66.67% - 72.00%
2× (non-verbose baseline)	<Query>\n<Query>	97.78%	97.33% - 98.67%

2× verbose	Paper’s exact template	95.08%	93.24% - 98.67%
3× verbose	Paper’s exact template	94.22%	92.00% - 96.00%
4× verbose	Self-built template	89.72%	86.67% - 94.67%

Following the paper’s exact template for 2× and 3× verbose, the mean accuracy is nearly identical. This does not confirm the paper’s claim that 3× often substantially outperforms 2×, at least on gemini-3.6-flash. The non-verbose 2× condition outperforms both verbose variants, signifying that connector phrases introduce more noise rather than benefit on this task. The simplest possible repetition format, a bare newline, outperforms all verbose variants across all runs.

The 4× verbose condition shows an accuracy decline, though the range overlaps with 3× at its upper end. The most likely explanation is a task-specific context-inflation effect: beyond a certain repetition level, additional copies of the name provide little new information while increasing the amount of redundant context the model must distinguish when performing positional retrieval. The 3× to 4× decline may reflect a threshold beyond which context length inflation will outweigh the benefit of additional repetition, potentially introducing off-by-one errors in position counting, attention dilution, or answer-selection errors. However, since the additional 4× connector is self-constructed, it is unclear whether the performance drop is caused by the longer context, the specific wording of the connector, or both.

This experiment was run only on NameIndex, and the paper’s claim may hold true on other tasks where context length inflation is less detrimental. It is worth noting that the scoreable n for one 2× verbose run and one 4× verbose run was 74, not 75 (see Appendix A.4).

4.3 Cross-model Comparisons

The paper’s Gemini models were gemini-2.0-flash and gemini-2.0-flash-lite, with the latter being used to emphasize the impact of prompt repetition for the NameIndex task. As previously mentioned, both models were unavailable as of August 2026. This experiment runs the

NameIndex task across six Gemini models: gemini-3.1-flash-lite, gemini-3.5-flash-lite, gemini-3.5-flash, gemini-3.6-flash, gemini-3.7-flash, and gemini-3.8-flash to see how the repetition effect changes across model generations. gemini-3.7-flash was run six times (runs 1-3 were excluded under the ERROR+BLOCKED threshold described in Section 2) and runs 4-6 are used as the primary result.

<u>Model</u>	<u>Mean Baseline</u>	<u>Range</u>	<u>Mean Repetition</u>	<u>Range</u>	<u>Mean Δ</u>	<u>Significant</u>	<u># Runs</u>
gemini-2.0-flash-lite (paper)	21.33%		97.33%		+76.00%	Yes	
gemini-3.1-flash-lite	70.67%	70.67% - 70.67%	100.00%	100.00% - 100.00%	+29.33%	Yes	3
gemini-3.5-flash-lite	45.33%	44.00% - 46.67%	95.56%	94.67% - 97.33%	+50.22%	Yes	3
gemini-3.5-flash	85.78%	84.00% - 89.33%	81.78%	80.00% - 84.00%	-4.00%	No	3
gemini-3.6-flash	68.89%	66.67% - 72.00%	97.78%	97.33% - 98.67%	+28.89%	Yes	3
gemini-3.7-flash	80.00%	76.00% - 84.00%	94.22%	90.67% - 97.33%	+14.22%	Yes	3
gemini-3.8-flash	82.22%	73.33% - 90.67%	95.56%	94.67% - 97.33%	+13.33%	Mixed (2 of 3)	3

For models gemini-3.1-flash-lite, gemini-3.5-flash-lite, and gemini-3.6-flash, prompt repetitions produce significant gains and repetition ceiling sits near or at 97-100%. These models follow an

expected pattern. gemini-3.1-flash-lite achieves a perfect repetition ceiling of 100% across all three runs with zero regressions (b=0, c=22 in every run), which is the largest observed ceiling. Temperature was set to 0 and gemini-3.1-flash-lite produced identical output across all three runs, so its zero-width range reflects determinism rather than sampling stability.

gemini-3.5-flash-lite shows the largest absolute gain (+50.22%), consistent with its lower baseline.

For gemini-3.5-flash, repetition slightly hurts performance across all three runs and none of the runs approach statistical significance ($p = 0.4545, 0.6476, 0.8318$). Under repetition, this model produced 4, 5, and 3 “EMPTY” responses across three runs, respectively. This indicates that the extractor found no name in the model’s raw output. The failure taxonomy resolves this, showing the regression is concentrated in output format rather than in a collapse of retrieval. Under repetition this model produced no bare wrong names at all (OTHER: $13 \rightarrow 0$ across three runs), but 29 of its 41 failures were leaked responses prefixed with a list index, and a further 12 were index-only emissions the extractor reduced to an empty string. The model shifted almost entirely into list-continuation mode under repetition, which is the opposite of what every other model in this study did. Why repetition triggers this mode on gemini-3.5-flash specifically is not resolved by this data.

gemini-3.7-flash requires attention because of its strong instability across the first three runs. Runs 1-3 produce various amounts of baseline and repetition “ERROR and BLOCKED” responses, ranging from 4-20, which is far outside the range of any other model or condition in this experiment. Runs 4-6, conducted after resolving the suspected VPN-caused API instability, produced substantially different results. When the extraction function operates, gemini-3.7-flash produces trends consistent with what the paper claims.

<u>Run</u>	<u>Baseline</u>	<u>Repetition</u>	<u>ERROR + BLOCKED (Baseline)</u>	<u>ERROR + BLOCKED (Repetition)</u>	<u>p-value</u>
1	53.33%	73.33%	20	17	0.0107

(excluded)					
2 (excluded)	72.00%	85.33%	6	6	0.0639
3 (excluded)	73.33%	88.00%	4	6	0.0347
4	80.00%	97.33%	0	0	0.0023
5	84.00%	94.67%	1	0	0.0768
6	76.00%	90.67%	2	0	0.0266

5 Limitations

5.1 Sample Size

ARC was tested at $n=100$, a sample size that may be underpowered to confidently establish a valid improvement. The McNemar test for question-first yields $p=0.1094$ with only 10 discordant pairs, above the critical value. The paper does not publish exact ARC figures, but its Figure 1 suggests that for Gemini 2.0 Flash Lite, ARC options-first produced approximately a 6% gain, which is consistent with this reproduction's result. The paper's Figure 1 shows no question-first gain for Gemini 2.0 Flash Lite, whereas this reproduction finds +6%, though neither result is significant.

5.2 Name List

The paper published one example NameIndex list in its appendix but does not specify whether this list was reused for different models or regenerated after every test. This reproduction did not use the provided name list. Instead names were generated by sampling from common US first

and last names with a fixed seed. Since the NameIndex task is positional retrieval rather than name recognition, this deviation is unlikely to be material, but it cannot be ruled out entirely.

5.3 gemini-3.8-flash

This model was tested across three runs conducted one day after its public release. Considering the mixed significance across three runs, the results may reflect instability in the model’s API behavior at the time of testing rather than a genuine property of the model. These results should be interpreted with corresponding caution and may not replicate once the model’s configuration stabilizes.

5.4 Scope

This reproduction tests two of the paper’s seven benchmarks and restricts model coverage to the Gemini family. No claims are made about whether the effect generalizes to OpenBookQA, GSM8K, MMLU-Pro, or MATH, to ChatGPT, Deepseek, or Claude-family models, or to the MiddleMatch task. The padding control ablation was not implemented, so the paper’s claim that gains are semantic rather than length-driven cannot be independently verified here.

5.5 Answer Extraction

The NameIndex extractor takes only the first line of the model’s response and strips leading punctuation. This design has known failure modes: it fails when the model produces reasoning preamble before the answer, when the response is empty or truncated, and when the model outputs a bare list index instead of a name. The last two appear in the gemini-3.5-flash repetition condition. When the model’s output continues into the next list item, the extractor returns the first name it finds, which in these cases is itself the wrong name (Section 5.6).

5.6 Leakage and Retrieval Failure Co-occur

Baseline responses frequently continued past the answer into the next item of the name list, producing raw output such as “Paul Scott\n 15. James”. These are genuine failures since the name given is wrong, and no extractor could recover the correct one (Section 5.7). What makes them worth separating is that they co-occur with retrieval failure at a high rate and disappear alongside it. Across gemini-3.7-flash runs 4-6, total failures fell from 45 to 13 under repetition,

and leaked responses account for 24 of that reduction of 32. The same holds for gemini-3.6-flash (41 of 65) and gemini-3.8-flash (23 of 30). This suggests a shared cause: in the baseline condition the model is frequently in list-continuation mode rather than answer-the-question mode, and that mode produces both the ragged output and the wrong name. Repetition appears to suppress both together, which is consistent with the paper's account rather than an alternative to it.

5.7 Extraction Bias

A reasonable concern is that the extractor discards correct answers present in the raw output, which would inflate the measured failure count in whichever condition produces messier text. This was tested directly. Across all 312 non-infrastructure failures in the cross-model experiment, the correct answer appeared anywhere in the raw model output exactly once. The extractor is therefore not rejecting correct retrievals, and a lenient scoring scheme that credited near-misses would change one item across the entire study. Responses categorized as “LEAKED” are genuine failures.

5.8 Leakage Is Specific to the Flash Family

Leaked output is not a uniform property of the task. In the baseline condition it is the dominant failure category for gemini-3.6-flash, gemini-3.7-flash, and gemini-3.8-flash, but occurs zero times across all 450 responses from gemini-3.1-flash-lite and zero times across all 450 responses from gemini-3.5-flash-lite. The lite models therefore reach their gains without ever entering list-continuation mode, while for the flash models that mode accounts for most baseline failures. The two families arrive at similar repetition ceilings by different routes.

5.9 Multiple Comparisons

All tests in this reproduction use the paper's $\alpha=0.1$ threshold without correction, which matches the paper's own methodology. However, since 23 McNemar tests were run across this study, the family-wise false-positive rate under that scheme is high, and a correction is applied here as a check on the primary result.

Among twenty-three McNemar tests that were run (two ARC orderings, three Mandarin, and three for each gemini model), seventeen were significant at the $\alpha=0.1$ threshold. The six that were not significant are both ARC orderings, gemini-3.8-flash run 2, and all three gemini-3.5-flash runs. When the Holm-Bonferroni step-down procedure is applied across all 23 tests at $\alpha=0.1$, fourteen of the seventeen remain significant. The procedure halts at gemini-3.7-flash run 6, whose $p = 0.0266$ exceeds its threshold of 0.0111. The two runs below it, gemini-3.8-flash run 1 and gemini-3.7-flash run 5, are retained.

6 Conclusion

The paper’s central claim reproduces; prompt repetition produced large and strongly significant accuracy gains on NameIndex across every Gemini model tested except one. The effect remains significant for the primary model for both English and Mandarin and for both lite models. Prompt repetition did not penalize latency; the median and mean response times were marginally lower for repetition than at baseline. On the ARC-Challenge, the direction favors repetition in both orderings but neither reaches significance, which is consistent with the paper’s finding that gains are slight for multiple-choice tasks where the baseline is already high. Outside of confirming the validity of the effect, the failure taxonomy shows it arrives by different routes on different models or families. On the two lite models, repetition eliminated ordinary wrong answers and nothing else; there is no list-continuation behaviour to suppress. On the flash models, most baseline failures came from ragged output that ran past the answer into the next list item, and repetition suppressed those and the wrong answers together. The paper reports accuracy alone and could not have found this distinction. This co-occurrence suggests that on flash-family models, part of what repetition does is move the model out of list-continuation mode and into answer-the-question mode, which is compatible with the paper’s bidirectional-attention account but not identical to it.

The exception is gemini-3.5-flash, being the only model where repetition reduced accuracy in all three runs. The composition of its failures is inverted relative to every other model. Under repetition, it produced no bare wrong names, but 29 of its 41 failures were index-prefixed responses while the other 12 were index-only emissions the extractor failed to parse. The reason

why repetition pushed this specific model into the failure mode repetition suppresses in every other model is unresolved.

Three potential gaps found in this reproduction could be worth addressing. First, logging output token counts would test the other half of the paper’s claim unexamined here. Second, the Mandarin experiment changed the instruction language only. Since the names themselves remained in English, running the task with both instruction and names in the same or different non-Latin language would test whether the effect can survive a full cross-lingual transfer. Third, adding a $1\times$ verbose or a $3\times$ non-verbose could separate repetition count from connector wording in 4.2, the scaling experiment, which is currently confounded.

References

Clark et al. "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge." arXiv:1803.05457, 2018.

Leviathan, Kalman, and Matias. "Prompt Repetition Improves Non-Reasoning LLMs." arXiv:2512.14982, December 2025.

McNemar, Q. "Note on the sampling error of the difference between correlated proportions or percentages." *Psychometrika*, 12(2):153–157, 1947.

Appendix A Failure Taxonomy

A.1 Why a Taxonomy Is Needed

Scoring a NameIndex response as simply correct or incorrect can discard information that turns out to matter. A response can fail because the API call never completed, because the provider withheld the output, because the model produced text the extractor could not parse, or because the model named the wrong person. These are different events with different implications, and collapsing them into a single incorrect count makes it impossible to tell whether an intervention improved the model’s retrieval or merely improved the shape of its output. Every failure in this study is therefore assigned to exactly one of five categories.

A.2 Category Definitions

ERROR. The API call did not complete and no model output was produced. These are identifiable by a recorded latency of exactly 0.0 seconds. An ERROR, being a property of the testing environment and not of the model, is the only category excluded from the denominator.

BLOCKED. The API call completed and the provider returned a block rather than generated text. Recorded latencies for these responses range from 1.08 to 19.56 seconds and are continuously distributed, which is consistent with a safety filter acting on generated output rather than a token-limit truncation, which would cluster at the long end. BLOCKED is scored as incorrect. The model received the prompt and generated tokens, and a response that cannot be delivered is a failed response.

EMPTY. The model returned text but the extractor produced no name from it. In practice this occurred when the model emitted a bare list index such as “21.” or “16.”, which the extractor reduced to an empty string after stripping punctuation. EMPTY is scored as incorrect.

LEAKED. The model produced a name and then continued past it into subsequent items of the name list, or prefixed its answer with a list index. Detection is by two tests on the raw output: the presence of a new line followed by further text, or a leading number followed by a period, colon, or closing parenthesis. Examples of LEAKED can be “Paul Scott\n 15. James” and “23. Mary Lewis”. The extractor salvages the first name, so a LEAKED response still yields a candidate answer, which is then compared against the correct answer in the usual way. LEAKED is scored as incorrect.

OTHER. The model produced a single well-formed name that does not match the correct answer. This is the only category that unambiguously represents a retrieval failure.

A.3 What LEAKED Does and Does Not Mean

LEAKED is the most easily misread category, so it is worth stating precisely. It does not mean the model retrieved the correct answer and the extractor lost it, it is a genuine failure (as mentioned in Section 5.7). The purpose of putting LEAKED into its own category is diagnostic rather than corrective. Leaked responses are roughly ten times more common in the baseline

condition than under repetition, and isolating them reveals that leaked output and retrieval failure share a common cause and are suppressed together.

An index-prefixed response such as “23. Mary Lewis” raises the possibility that the model is answering a different item in the list rather than producing malformed output. This was tested. Of 44 index-prefixed responses, none contained the correct answer to the question named in the prefix, at that index or at the adjacent indices. The prefix therefore carries no positional information and is simply leftover list formatting.

A.4 The Scoreable Denominator

Accuracy is reported over a scoreable denominator equal to 75 minus the number of ERROR responses. Because McNemar’s test is a paired test, the same questions must be present in both conditions, so a question is dropped from both arms if either arm errored on it, rather than from each arm independently. Dropping ERRORS per condition would leave the two arms scored over different question sets and would make the paired test invalid. In practice this affects only two runs of the repetition-count experiment, each of which contains a single ERROR and is scored over 74 rather than 75; every other reported run has a denominator of 75.

A.5 Run Selection

Runs were excluded by a single pre-specified threshold: a run is discarded if either condition produced five or more combined ERROR and BLOCKED responses, and the first three runs passing that threshold constitute the reported result. Applied to the full set of runs, this rule discards gemini-3.7-flash runs 1, 2, and 3, which produced 20, 6, and 6 such responses in their worst condition respectively, and retains runs 4, 5, and 6, which produced 0, 1, and 2. No other model produced any run above the threshold, so every other model is reported on its first three runs.

A.6 Reading the Failure Taxonomy Tables

Each row of the taxonomy tables reports one model, one run, and one condition. The five category columns sum to the Total Fail column. Total n is always 75; Scoreable n is 75 minus ERROR; and the percentage correct is the number of correct responses divided by Scoreable n.